CÓDIGOCLARO · RUTA KOTLIN

# Tema 13 · Coroutines

Cuaderno de estudio para DAM/DAW · 10 lecciones

## Qué vas a aprender

suspend, launch, async, scopes y cancelación.

Este cuaderno trabaja coroutines en pasos cortos. La meta no es repetir palabras de memoria: es reconocer el problema, elegir una herramienta adecuada, explicar por qué funciona y comprobarla con datos distintos. Reserva una sesión para leer el modelo, otra para resolver el ejercicio y una tercera para intentar el laboratorio sin mirar la solución.

### Resultados de aprendizaje

- Explicar con tus palabras qué problema resuelve coroutines y en qué parte de una aplicación DAM/DAW aparece.
- Leer el ejemplo, predecir el resultado y justificar cada decisión relevante.
- Resolver una práctica, comprobar casos normales y límite, y depurar un fallo con un método reproducible.

### Cómo estudiar este tema

Ten a mano un editor compatible con la tecnología y copia los ejemplos en un archivo nuevo. Antes de ejecutar, escribe tu predicción. Si no coincide con la salida, no borres el intento: anota qué supuesto era incorrecto y prueba una variación mínima. Al terminar, explica el concepto a otra persona usando un ejemplo propio.

### Punto de control inicial

Antes de avanzar, escribe dos ideas que ya conoces sobre este tema y una pregunta que todavía no sabes responder. Vuelve a ellas al final de la lección 10. Si cambias de respuesta, explica qué ejemplo o prueba hizo que cambiaras de opinión.

# Lección 1 · El modelo mental

## Idea central

suspend marca una función que puede ceder y reanudarse mediante operaciones suspendibles; por sí sola no crea un hilo ni inicia concurrencia. kotlinx.coroutines aporta builders como launch y un CoroutineScope que gestiona hijos y cancelación. La cancelación es cooperativa: cancelar un Job padre se propaga a sus hijos. Los bloques finally se ejecutan durante la cancelación para liberar recursos; cancelAndJoin solicita la cancelación y espera a que termine el Job y sus hijos. delay y awaitCancellation suspenden sin bloquear el hilo.

Para razonar sobre coroutines, separa cuatro preguntas: qué datos existen al empezar, qué regla se aplica, qué cambia durante el proceso y qué resultado debe observar quien usa el programa. Esta separación ayuda a no confundir la sintaxis con la finalidad. Una línea puede compilar y aun así resolver mal el problema si el dato representa otra cosa o falta un caso.

## En Kotlin, ten presente este marco

El itinerario usa Kotlin/JVM: el compilador infiere muchos tipos, pero el programa sigue teniendo comprobación estática. val impide reasignar el nombre, aunque el objeto referenciado pueda ser mutable; var permite reasignar. Los tipos nullable se escriben con ? y obligan a tratar la ausencia. La biblioteca distingue interfaces de solo lectura de interfaces mutables.

## Comprueba tu comprensión

- ¿Qué parte de la explicación define la regla principal de coroutines?
- ¿Qué dato puede cambiar y qué dato debería permanecer estable?
- ¿Qué observación concreta demostraría que tu programa cumple el objetivo?

# Lección 2 · Vocabulario y reglas

## Reglas de Kotlin que afectan a este tema

suspend marca una función que puede suspenderse, pero no inicia por sí sola una coroutine. launch crea trabajo sin resultado directo; async produce un Deferred que se consulta con await. El alcance enlaza trabajo con su ciclo de vida; cancelar el padre debe propagar cancelación a sus tareas hijas.

## Ideas específicas de coroutines

La asincronía permite esperar a una operación sin bloquear el flujo principal; no vuelve instantánea la operación ni garantiza que termine bien. Define qué ocurre en éxito, error, cancelación y respuesta tardía. Evita actualizar la interfaz con una respuesta antigua después de que el usuario haya cambiado de pantalla o de consulta. Propaga los errores hasta el punto que pueda mostrarlos o recuperarse.

## Glosario de trabajo

| Término | Significado en este recorrido |
| --- | --- |
| Operación pendiente | Trabajo que todavía no ha devuelto un resultado. |
| Await | Punto donde el flujo espera el resultado de una tarea asincrónica. |
| Cancelación | Solicitud para dejar de esperar o detener trabajo que ya no se necesita. |
| Carrera | Resultado dependiente del orden temporal de operaciones concurrentes. |

Relaciona cada término con una línea del ejemplo o una decisión del ejercicio. Comprueba las diferencias de versión en la fuente oficial enlazada antes de aplicar una regla al proyecto.

# Lección 3 · Lectura guiada del ejemplo

Primero identifica la entrada, luego sigue el valor por el programa. No supongas que una asignación transforma el dato: a veces solo enlaza un nombre con el mismo objeto. Observa dónde empieza cada bloque, qué condición activa una rama y qué instrucción produce la salida. Si el ejemplo tiene varias líneas, ejecútalo por partes y añade temporalmente observaciones seguras para ver los valores intermedios.

Código · kotlin

```kotlin
import kotlinx.coroutines.*
fun main() = runBlocking {
    val iniciado = CompletableDeferred<Unit>()
    val padre = launch {
        try {
            launch {
                try { iniciado.complete(Unit); awaitCancellation() }
                finally { println("Hijo ejecuta finally") }
            }
            awaitCancellation()
        } finally { println("Padre ejecuta finally") }
    }
    iniciado.await()
    padre.cancelAndJoin()
    println("Padre e hijo terminan")
}
```

Contexto: Requiere kotlinx-coroutines-core; runBlocking, launch, CompletableDeferred y cancelAndJoin pertenecen a esa biblioteca.

## Salida esperada

```
Se muestran ambos mensajes finally (su orden relativo no se fija). Después aparece
«Padre e hijo terminan», porque cancelAndJoin espera a que concluya también el hijo.
```

## Registro de la traza

| Momento | Qué anotar |
|---|---|
| Antes de ejecutar | Valores iniciales, sus tipos y qué operación esperas que ocurra. |
| Después de cada paso | Nombre afectado, valor nuevo y regla que produjo el cambio. |
| Al encontrar una decisión | Condición evaluada y camino que se ejecutó. |
| Al terminar | Salida real, diferencia respecto a tu predicción y causa. |
| Con una entrada distinta | Qué resultado cambia y qué parte del contrato permanece igual. |

## Traza en papel

- Antes de la primera instrucción: apunta los valores iniciales y sus tipos.
- Después de cada operación: anota solo los nombres que hayan cambiado y la razón.
- Al final: compara tu predicción con «Se muestran ambos mensajes finally (su orden relativo no se fija). Después aparece «Padre e hijo terminan», porque cancelAndJoin espera a que concluya también el hijo.» y explica la diferencia si la hay.

Cambia un dato de entrada y vuelve a hacer la traza. Una buena explicación debe anticipar el nuevo resultado, no limitarse a repetir la salida original.

# Lección 4 · Decidir cuándo usarlo

La asincronía permite esperar a una operación sin bloquear el flujo principal; no vuelve instantánea la operación ni garantiza que termine bien. Define qué ocurre en éxito, error, cancelación y respuesta tardía. Evita actualizar la interfaz con una respuesta antigua después de que el usuario haya cambiado de pantalla o de consulta. Propaga los errores hasta el punto que pueda mostrarlos o recuperarse.

## Caso de uso

Una pantalla cancela una operación con tareas hijas al cerrarse y necesita comprobar que todas liberaron sus recursos.

## Compara antes de elegir

- ¿La solución más directa deja claro el propósito o requiere conocer demasiados detalles internos?
- ¿Qué coste aparece si crece el volumen de datos o se añade un nuevo caso?
- ¿Qué comprobación automatizada detectaría una modificación accidental?

## Enfoques relacionados

- Llamar a padre.cancel() y después padre.join() para separar la solicitud de cancelación de la espera.
- Usar una señal CompletableDeferred o un canal de prueba para coordinar que el hijo ya está suspendido antes de cancelar.

No existe una alternativa mejor en todos los contextos. Elige una y escribe su ventaja y su límite para el tamaño, la versión y los requisitos del ejercicio. Si dos enfoques cumplen exactamente el mismo contrato y ambos son legibles, pueden considerarse válidos.

# Lección 5 · Errores y diagnóstico

## Fallos que conviene reconocer

- Afirmar que suspend mueve automáticamente el trabajo a otro hilo. Para localizarlo, reduce el caso a la menor entrada que lo reproduce, observa el valor implicado y verifica la regla antes de cambiar el código.
- Lanzar tareas en un scope global que no se cancela con quien las necesita. Para localizarlo, reduce el caso a la menor entrada que lo reproduce, observa el valor implicado y verifica la regla antes de cambiar el código.
- Suponer que cancel() espera a que termine el Job; usa cancelAndJoin() cuando debas esperar, y no ocultes la cancelación capturando CancellationException. Para localizarlo, reduce el caso a la menor entrada que lo reproduce, observa el valor implicado y verifica la regla antes de cambiar el código.

## Rutina de depuración

1. Describe el resultado esperado y el resultado real sin mezclar ambos. 2. Reproduce el fallo con una entrada fija. 3. Reduce el programa hasta dejar la primera operación que se comporta distinto. 4. Formula una hipótesis comprobable y cambia una sola cosa. 5. Añade una prueba que impida que el mismo fallo vuelva. Este orden también funciona cuando el error aparece en una API, una interfaz o un archivo.

## Preguntas de revisión

- ¿El error está en los datos, en la regla o en cómo se presenta el resultado?
- ¿Qué caso límite no estaba cubierto por el ejemplo inicial?
- ¿La solución corrige la causa o solo oculta el síntoma?

# Lección 6 · Práctica guiada

## Situación de DAM/DAW

Una pantalla cancela una operación con tareas hijas al cerrarse y necesita comprobar que todas liberaron sus recursos.

## Enunciado

Crea un Job padre con un hijo que espera mediante awaitCancellation(). Usa CompletableDeferred para saber que el hijo ha empezado; cancela el padre con cancelAndJoin() y registra un mensaje en finally en ambos. Verifica la propagación y que los dos finally ocurren antes del mensaje posterior, sin exigir un orden entre ellos.

## Convierte el enunciado en requisitos verificables

- Escribe qué entradas acepta la solución y qué debe devolver o mostrar.
- Enumera las reglas de negocio y separa las que son obligatorias de las decisiones de presentación.
- Antes de programar, completa la tabla de pruebas y predice cada resultado.

## Casos que debe superar

| Entrada | Resultado esperado |
|---|---|
| El hijo completa la señal y después se cancela el padre | El hijo recibe la cancelación y ejecuta su finally. |
| El Job padre entra en cancelación | El bloque finally del padre también se ejecuta. |
| Esperar con padre.cancelAndJoin() | Cuando retorna, ambos finally han ocurrido; el mensaje posterior se imprime después. |

## Qué demuestra cada caso

Clasifica cada fila de la tabla como caso habitual, límite o variación importante. Si el enunciado no define qué hacer con una entrada fuera de contrato, no inventes un resultado silencioso: anota la ambigüedad y propone una regla antes de programar. En la práctica profesional, acordar esa regla evita que dos implementaciones parezcan correctas y se comporten de forma incompatible.

## Plan de solución

- Divide la tarea en pasos pequeños que puedan comprobarse por separado.
- Implementa primero el caso habitual y después añade los límites y errores previstos.
- Ejecuta cada caso tras un cambio; no esperes al final para descubrir qué parte falló.

# Lección 7 · Solución explicada

Una solución no se justifica solo porque la salida coincida en un ejemplo. Debe respetar el contrato para todas las entradas admitidas. Lee el código siguiente y vincula cada operación con una frase del enunciado. Si no puedes señalar esa relación, revisa el diseño antes de continuar.

Código · kotlin

```
import kotlinx.coroutines.*
fun main() = runBlocking {
    val iniciado = CompletableDeferred<Unit>()
    val padre = launch {
        try {
            launch {
                try { iniciado.complete(Unit); awaitCancellation() }
                finally { println("Hijo ejecuta finally") }
            }
            awaitCancellation()
        } finally { println("Padre ejecuta finally") }
    }
    iniciado.await()
    padre.cancelAndJoin()
    println("Padre e hijo terminan")
}
Criterio: aparecen ambos mensajes finally antes del mensaje final; no se exige
orden entre padre e hijo.
```

## Por qué se sigue este razonamiento

El hijo se crea dentro del cuerpo del padre y hereda su Job. La señal evita cancelar antes de que el hijo empiece a esperar. Al cancelar el padre, la cancelación llega al hijo; cada finally registra la limpieza. cancelAndJoin no retorna hasta que el padre y sus hijos completan, por lo que el mensaje final sirve como comprobación. El orden entre los finally no es parte del contrato.

## Comprobación de corrección

Comprueba cada caso de la tabla y explica qué parte de la solución lo cubre. Después revisa el caso mínimo, el máximo razonable y una entrada inválida si el contrato la permite. Comprueba también los tipos de entrada y salida: una coincidencia textual puede ocultar un valor mal tipado o una conversión con pérdida.

| Paso | Justificación que debes poder dar |
|---|---|
| Contrato | Qué valores acepta el ejercicio y qué resultado promete. |

| | |
|---|---|
| Regla | Qué operación transforma la entrada en la salida. |
| Pruebas | Qué caso comprueba lo normal y cuál descubre un límite. |

### Alternativas válidas

- Llamar a padre.cancel() y después padre.join() para separar la solicitud de cancelación de la espera.
- Usar una señal CompletableDeferred o un canal de prueba para coordinar que el hijo ya está suspendido antes de cancelar.

Al comparar soluciones, prioriza contrato correcto, legibilidad y pruebas. Una alternativa que cambia la representación interna puede ser válida; una que cambia silenciosamente el significado de la salida no lo es.

## Lección 8 · Laboratorio de aplicación

Retoma el ejercicio de coroutines y amplíalo como una pequeña funcionalidad de una aplicación de gestión académica. Conserva el requisito original y añade una regla nueva que tenga sentido para los datos. Antes de modificar el código, escribe la regla en lenguaje natural, un ejemplo que la cumpla y otro que la incumpla.

### Variación para transferir lo aprendido

Añade otro hijo con su propio finally y verifica que cancelAndJoin retorna solo después de la limpieza de los dos hijos.

### Desafíos progresivos

- Nivel 1: añade un caso límite que no aparecía en la tabla y registra el resultado esperado.
- Nivel 2: cambia la forma de recibir o recorrer los datos sin cambiar el contrato externo.
- Nivel 3: separa la regla principal de su entrada/salida y prueba ambas partes de forma independiente.

### Criterios de entrega

- La solución conserva el resultado esperado del ejercicio original.
- El nuevo comportamiento queda cubierto por una prueba que falla antes del cambio.

- Los nombres permiten seguir el flujo sin comentarios que repitan la sintaxis.
- La entrada inválida o el error externo tiene una respuesta definida.

### Prueba de regresión

Guarda los casos originales antes de añadir la regla nueva. Ejecuta la batería completa después de cada cambio, no solo el nuevo caso. Si una prueba deja de pasar, decide si encontraste un defecto o si cambió el contrato y hay que actualizar la expectativa con una explicación. Termina con una nota breve: qué cambió, qué comprobaste y qué límite sigue abierto.

Anota qué parte se puede reutilizar en otro módulo y qué parte depende del contexto de la aplicación. Esa distinción ayuda a decidir si merece la pena extraer una función o componente.

## Lección 9 · Síntesis y fuentes

### Resumen del tema

Has estudiado coroutines desde el modelo conceptual hasta una práctica comprobable. La secuencia de trabajo es: identificar entradas y contrato, elegir la operación adecuada, seguir el flujo, comprobar casos límite y conservar una prueba para la siguiente modificación. Aplica las reglas de Kotlin sin confundir una característica del lenguaje con una API concreta del entorno.

### Lista de control antes de avanzar

- Puedo explicar el concepto con un ejemplo propio y predecir su salida.
- Puedo identificar al menos un caso límite y un error habitual.
- Puedo comparar dos enfoques y justificar mi elección.
- Puedo verificar una solución sin depender de mirar el resultado una sola vez.

### Fuentes oficiales para ampliar

JetBrains / Kotlin — Coroutines basics · JetBrains / kotlinx.coroutines — Coroutines guide · JetBrains / Cancellation and timeouts

Consulta la sección de la fuente que corresponda a la versión de tu proyecto. Este cuaderno resume conceptos para estudiar; cuando haya una diferencia de detalle, prevalece la documentación oficial del lenguaje o biblioteca.

# Lección 10 · Repaso activo

Cierra el material y responde sin copiar frases. Si dudas, vuelve a la página indicada, comprueba la fuente primaria y reescribe la respuesta con tus palabras. La recuperación activa ayuda a detectar qué parte aún depende de memorizar un ejemplo concreto.

## Preguntas

- ¿Qué problema aborda coroutines y qué dato necesita?
- ¿Qué diferencia hay entre una solución que funciona en el ejemplo y una que cumple el contrato?
- ¿Qué error frecuente te parece más probable y cómo lo reproducirías?
- ¿Qué alternativa elegirías en un proyecto pequeño y por qué?

## Respuestas de comprobación

- La idea clave se resume en esta explicación: suspend marca una función que puede ceder y reanudarse mediante operaciones suspendibles; por sí sola no crea un hilo ni inicia concurrencia. kotlinx.coroutines aporta builders como launch y un CoroutineScope que gestiona hijos y cancelación. La cancelación es cooperativa: cancelar un Job padre se propaga a sus hijos. Los bloques finally se ejecutan durante la cancelación para liberar recursos; cancelAndJoin solicita la cancelación y espera a que termine el Job y sus hijos. delay y awaitCancellation suspenden sin bloquear el hilo.
- Una fuente inicial de fallo descrita en el tema es: Afirmar que suspend mueve automáticamente el trabajo a otro hilo.
- Una alternativa es válida si mantiene las entradas, salidas y reglas acordadas y supera las mismas pruebas.